

МИФИ — Оркестрация и контейнеризация

Sentiment API (Java) в Kubernetes (Minikube) + HPA + мониторинг Prometheus/Grafana



Учебный проект (локальный кластер)

Фокус: развёртывание • autoscaling • observability

Введение

Что и зачем было сделано

Цель проекта

Развернуть микросервис для анализа тональности в Kubernetes.
Сделать сервис наблюдаемым (метрики) и масштабируемым (HPA).
Продemonстрировать полный цикл: контейнер → манифесты → мониторинг.

Ключевые результаты

3 реплики приложения + Service + Ingress.
HPA: 3...10 реплик по CPU (target 50%).
Prometheus/Grafana через kube-prometheus-stack.
Собственные метрики Spring Boot через /actuator/prometheus.

Что демонстрируем на защите

- доступ к API через Ingress: /api/sentiment и /health;
- рост нагрузки → рост реплик (kubectl get hpa -w);
- сбор метрик в Prometheus и визуализация в Grafana;
- связь с современными трендами (arXiv 2024–2025): AI-driven autoscaling и оптимизация инференса.

Используемый стек и среда выполнения

Компоненты и окружение из отчёта

Стек



Docker

Сборка образа sentiment-app и запуск контейнера



Kubernetes (Minikube)

Deployment/Service/Ingress/HPA в локальном кластере



Helm

Установка kube-prometheus-stack



Prometheus

Сбор метрик через ServiceMonitor (PromQL)



Grafana

Дашборды HPA / Pods / JVM / HTTP

Среда выполнения



Ubuntu 22.04 (VM)

Запуск в VMware Workstation; ресурсы VM: 4 CPU / 5.5 GB RAM.

ОС хоста: Windows 11

Java 21 + Spring Boot 3 + Maven

Minikube: addons ingress + metrics-server

Ingress NGINX + minikube tunnel (LoadBalancer)

Архитектура решения

Потоки: входящий трафик • метрики • autoscaling



Легенда: вход → Ingress → Service → Pods; метрики → Prometheus → Grafana; HPA масштабирует Deployment

Компоненты решения

Кратко о каждом блоке

Sentiment API (Java)

Endpoints: /api/sentiment, /health

Метрики: /actuator/prometheus

Порт 8080

HPA

3...10 реплик

Target CPU utilization: 50%

Metrics Server (addon)

Deployment

3 реплики по умолчанию

Liveness/Readiness probes

Requests/Limits по CPU

Prometheus

kube-prometheus-stack

ServiceMonitor для приложения

Сбор системных и app-метрик

Service + Ingress

Service (LoadBalancer) + minikube tunnel

Ingress NGINX: /api и /health

Grafana

Готовые дашборды k8s/HPA

Панели JVM/HTTP

Explore (PromQL)

Развёртывание

Создание кластера и подготовка окружения

Этапы создания кластера (Minikube)

- Установка: Docker, kubectl, minikube, helm.
- Запуск кластера: `minikube start --cpus=4 --memory=5632`.
- Включение аддонов: `ingress`, `metrics-server`.
- Публикация Service типа `LoadBalancer`: `minikube tunnel`.

```
bereza@k8s-ubuntu:~$ minikube start --driver=docker --cpus=4 --memory=8192mb --nodes=2
🐳 minikube v1.37.0 on Ubuntu 22.04
🔧 Using the docker driver based on user configuration
🔧 Using Docker driver with root privileges
🔥 Starting "minikube" primary control-plane node in "minikube" cluster
📦 Pulling base image v0.0.48 ...
🔥 Creating docker container (CPUs=4, Memory=8192MB) ...
📦 Preparing Kubernetes v1.34.0 on Docker 28.4.0 ...
🔗 Configuring CNI (Container Networking Interface) ...
🔧 Verifying Kubernetes components...
  ▪ Using image gcr.io/k8s-minikube/storage-provisioner:v5
🌟 Enabled addons: storage-provisioner, default-storageclass

🔥 Starting "minikube-m02" worker node in "minikube" cluster
📦 Pulling base image v0.0.48 ...
🔥 Creating docker container (CPUs=4, Memory=8192MB) ...
🌐 Found network options:
  ▪ NO_PROXY=192.168.49.2
📦 Preparing Kubernetes v1.34.0 on Docker 28.4.0 ...
  ▪ env NO_PROXY=192.168.49.2
🔧 Verifying Kubernetes components...
🎉 Done! kubectl is now configured to use "minikube" cluster and "default" namespace by default
bereza@k8s-ubuntu:~$
```

```
bereza@k8s-ubuntu:~$ minikube status
```

```
minikube
type: Control Plane
host: Running
kubelet: Running
apiserver: Running
kubeconfig: Configured
```

```
minikube-m02
type: Worker
host: Running
kubelet: Running
```

```
bereza@k8s-ubuntu:~$ kubectl get nodes -o wide
```

NAME	STATUS	ROLES	AGE	VERSION	INTERNAL-IP	EXTERNAL-IP	OS-IMAGE	KERNEL-VERSION	CONTAINER-RUNTIME
minikube	Ready	control-plane	13m	v1.34.0	192.168.49.2	<none>	Ubuntu 22.04.5 LTS	5.15.0-164-generic	docker://28.4.0
minikube-m02	Ready	<none>	12m	v1.34.0	192.168.49.3	<none>	Ubuntu 22.04.5 LTS	5.15.0-164-generic	docker://28.4.0

```
bereza@k8s-ubuntu:~$ kubectl get pods -A
```

NAMESPACE	NAME	READY	STATUS	RESTARTS	AGE
kube-system	coredns-66bc5c9577-ffwb4	1/1	Running	2 (12m ago)	13m
kube-system	etcd-minikube	1/1	Running	0	13m
kube-system	kindnet-qwf56	1/1	Running	0	12m
kube-system	kindnet-zpqfj	1/1	Running	0	13m
kube-system	kube-apiserver-minikube	1/1	Running	0	13m
kube-system	kube-controller-manager-minikube	1/1	Running	0	13m
kube-system	kube-proxy-6p44d	1/1	Running	0	13m
kube-system	kube-proxy-gj4bq	1/1	Running	0	12m
kube-system	kube-scheduler-minikube	1/1	Running	0	13m
kube-system	storage-provisioner	1/1	Running	1 (12m ago)	13m

```
bereza@k8s-ubuntu:~$
```

Развёртывание

Применение манифестов и проверка

Манифесты Kubernetes

Deployment: 3 replicas + probes + requests/limits

Service: LoadBalancer (minikube tunnel)

Ingress: правила /api и /health

HPA: min=3, max=10, cpu=50%

Проверка после apply

kubectl get pods,svc,ingress,hpa

curl http://<host>/api/sentiment (POST JSON)

curl http://<host>/health

kubectl describe hpa (текущие метрики)

```
bereza@k8s-ubuntu:~/final-k8s-sentiment-template$ kubectl apply -f k8s/deployment.yaml
kubectl apply -f k8s/service.yaml
kubectl apply -f k8s/ingress.yaml
kubectl apply -f k8s/hpa.yaml
deployment.apps/sentiment-app created
serviceaccount/sentiment-sa created
service/sentiment-service created
ingress.networking.k8s.io/sentiment-ingress created
horizontalpodautoscaler.autoscaling/sentiment-hpa created
bereza@k8s-ubuntu:~/final-k8s-sentiment-template$ kubectl get deployments
kubectl get pods -o wide
kubectl get svc
kubectl get ingress
kubectl get hpa
NAME          READY   UP-TO-DATE   AVAILABLE   AGE
sentiment-app 3/3      3            3           20s
NAME          READY   STATUS    RESTARTS   AGE   IP            NODE          NOMINATED NODE   READINESS GATES
sentiment-app-58684d8c86-ctk9z 1/1     Running   0          20s   10.244.1.10   minikube-m02  <none>           <none>
sentiment-app-58684d8c86-fqkzd 1/1     Running   0          20s   10.244.0.8    minikube      <none>           <none>
sentiment-app-58684d8c86-h5zvk 1/1     Running   0          20s   10.244.1.11   minikube-m02  <none>           <none>
NAME          TYPE          CLUSTER-IP   EXTERNAL-IP   PORT(S)          AGE
kubernetes    ClusterIP     10.96.0.1    <none>        443/TCP          129m
sentiment-service LoadBalancer 10.105.164.4 <pending>      80:30657/TCP    20s
NAME          CLASS   HOSTS          ADDRESS          PORTS   AGE
sentiment-ingress nginx   sentiment.local 192.168.49.2     80      21s
NAME          REFERENCE          TARGETS          MINPODS   MAXPODS   REPLICAS   AGE
sentiment-hpa Deployment/sentiment-app cpu: <unknown>/50% 3          10        3           20s
bereza@k8s-ubuntu:~/final-k8s-sentiment-template$
```

```
bereza@k8s-ubuntu:~$ minikube tunnel
[sudo] password for bereza:
Status:
  machine: minikube
  pid: 71222
  route: 10.96.0.0/12 -> 192.168.49.2
  minikube: Running
  services: [sentiment-service]
errors:
  minikube: no errors
  router: no errors
  loadbalancer emulator: no errors
```

```
bereza@k8s-ubuntu:~$ curl "http://sentiment.local/api/sentiment?text=hello"
{"text":"hello","sentiment":"neutral","score":0.85}bereza@k8s-ub
bereza@k8s-ubuntu:~$ curl "http://sentiment.local/health"
{"status":"UP"}bereza@k8s-ubuntu:~$
```

Автомасштабирование (HPA)

Нагрузка → рост реплик

Настройка HPA

Target: CPU utilization = 50%

Replicas: 3...10

Источник метрик: metrics-server
(addon)

Наблюдение: `kubectl get hpa -w`

```
○ bereza@k8s-ubuntu:~$ kubectl get hpa sentiment-hpa -w
NAME          REFERENCE          TARGETS      MINPODS  MAXPODS  REPLICAS  AGE
sentiment-hpa Deployment/sentiment-app  cpu: 32%/50%  3         10         3         22m
sentiment-hpa Deployment/sentiment-app  cpu: 30%/50%  3         10         3         23m
sentiment-hpa Deployment/sentiment-app  cpu: 35%/50%  3         10         3         24m
sentiment-hpa Deployment/sentiment-app  cpu: 59%/50%  3         10         3         25m
```

```
○ bereza@k8s-ubuntu:~$ kubectl get pods -l app=sentiment-app -w
NAME                                READY   STATUS    RESTARTS   AGE
sentiment-app-58684d8c86-ctk9z      1/1     Running   0          23m
sentiment-app-58684d8c86-fqkzd      1/1     Running   0          23m
sentiment-app-58684d8c86-h5zvz      1/1     Running   0          23m
sentiment-app-58684d8c86-jvkvb      0/1     Pending   0          0s
sentiment-app-58684d8c86-jvkvb      0/1     Pending   0          0s
sentiment-app-58684d8c86-jvkvb      0/1     ContainerCreating  0          1s
sentiment-app-58684d8c86-jvkvb      0/1     Running   0          3s
sentiment-app-58684d8c86-jvkvb      1/1     Running   0          11s
```

Нагрузочное тестирование

Инструменты: ApacheBench (ab) / curl-циклы.

Нагрузка повышает CPU → HPA увеличивает replicas.

После стабилизации нагрузки — постепенный scale down.

```
● bereza@k8s-ubuntu:~$ cd ~/final-k8s-sentiment-template/app
● bereza@k8s-ubuntu:~/final-k8s-sentiment-template/app$ kubectl run load-generator --image=busybox --restart=Never -- /bin/sh -c 'while true; do wget -q -O- http://sentiment-service/api/sentiment?text=load; done'
pod/load-generator created
● bereza@k8s-ubuntu:~/final-k8s-sentiment-template/app$ kubectl run load-generator-2 --image=busybox --restart=Never -- /bin/sh -c 'while true; do wget -q -O- http://sentiment-service/api/sentiment?text=load; done'
pod/load-generator-2 created
● bereza@k8s-ubuntu:~/final-k8s-sentiment-template/app$ kubectl run load-generator-3 --image=busybox --restart=Never -- /bin/sh -c 'while true; do wget -q -O- http://sentiment-service/api/sentiment?text=load; done'
pod/load-generator-3 created
● bereza@k8s-ubuntu:~/final-k8s-sentiment-template/app$ kubectl get hpa sentiment-hpa
NAME          REFERENCE          TARGETS      MINPODS  MAXPODS  REPLICAS  AGE
sentiment-hpa Deployment/sentiment-app  cpu: 46%/50%  3         10         4         28m
○ bereza@k8s-ubuntu:~/final-k8s-sentiment-template/app$
```


Мониторинг

Prometheus + Grafana (kube-prometheus-stack)



Prometheus

Сбор метрик приложения через ServiceMonitor:

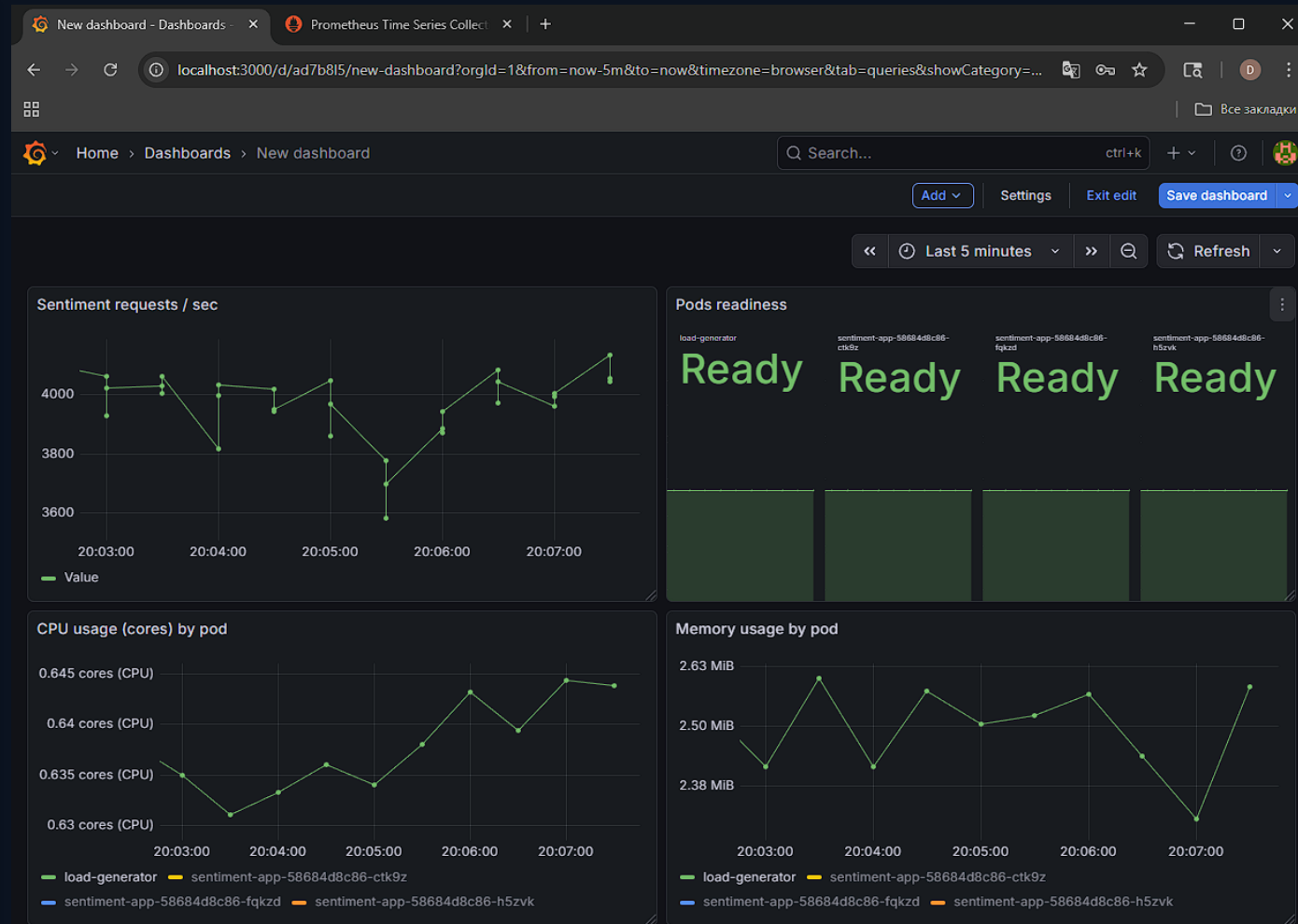
- Spring Boot Actuator: /actuator/prometheus
- scrape через ServiceMonitor (label selectors)
- kube-state-metrics + node-exporter — системные метрики



Grafana

Дашборды (готовые + кастомные):

- Kubernetes / Nodes / Pods
- HPA (replicas, CPU)
- JVM + HTTP метрики приложения



Анализ трендов (arXiv, 2024–2025)

AI-driven autoscaling и устойчивость

[1] HPA Multi-Agent System (2025)

Идея: декомпозиция цели «устойчивость» на failure-specific подцели, за которые отвечают агенты.

Дизайн-фреймворк: digital twin → обучение в симуляции → анализ поведения → перенос в кластер.

Вывод для проекта: следующий шаг после threshold-based HPA — ML/RL-скейлер с расширенными метриками.

[2] BAScaler (2024)

Burst-aware autoscaling: предсказание периодических всплесков + детекция «настоящих» бурстов.

Комбинация: ML для burst detection + RL для корректировки оценок ресурсов в «не бурст» режимах.

Практика: можно улучшить HPA, добавив предиктивную компоненту (RPS/latency/errors) поверх Prometheus.

Связь с проектом: текущий HPA по CPU = базовый уровень. Потенциал развития — multi-metric + предсказание нагрузки + RL-контроллер.

Анализ трендов (arXiv, 2024–2025)

Инференс LLM, GPU и cold-start

[3] AIBrix (2025)

Cloud-native фреймворк для LLM-инференса: LLM-specific autoscalers, prefix-aware routing, distributed KV cache.

Гибридная оркестрация: Kubernetes (coarse) + Ray (fine-grained) для multi-node serving.

Идея для проекта: при переходе от rule-based анализа к LLM — потребуется оптимизация холодных стартов и кэшей.

[4] KIS-S (2025)

GPU-aware simulator + PPO-скейлер для inference workloads.

Учёт GPU-метрик и P95 latency вместо только CPU.

Для проекта: следующий уровень — метрики latency + GPU (если появятся), обучение политики в симуляции.

[5] Cold starts + MPC (2025)

Проактивное планирование serverless: прогноз вызовов → prewarming + dispatching.

Цель: снизить tail-latency и расход ресурсов.

Для проекта: при Knative/KServe можно применить идеи prewarming для инференса моделей.

Практический вывод: observability (Prometheus/Grafana) — база для перехода к AI-driven autoscaling и более «умному» scheduling.

Заключение

Итоги, трудности, перспективы

Основные достижения

Полный пайплайн: Docker → Kubernetes → Ingress → HPA.
Наблюдаемость: Prometheus + Grafana + ServiceMonitor.
Готовность к расширению: multi-metric scaling, более сложные модели анализа тональности.

Трудности и решения

LoadBalancer в Minikube → minikube tunnel.
Метрики для HPA → включён metrics-server addon.
Сбор метрик приложения → Actuator + корректный ServiceMonitor/labels.

Перспективы (дальнейшее развитие)

- 1) Перейти к multi-metric autoscaling: CPU + latency + RPS + error rate (PromQL).
- 2) Экспериментировать с предиктивным scaling (ML/RL) на основе собранных метрик.
- 3) Подготовить перенос в managed Kubernetes (GKE/EKS/AKS): storage, ingress, secrets.
- 4) Вариант «в будущее»: замена rule-based логики на ML/LLM и оптимизация cold-start (Knative/KServe).